

Session – 10

TASK 1: Inode Structures, Hard Links and Symbolic Links

Question

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Create a folder and a file

```
ganesh@DESKTOP-79N3772:~$ mkdir -p ~/session10
~/session10ganesh@DESKTOP-79N3772:~$ cd ~/session10
ganesh@DESKTOP-79N3772:~/session10$ echo "Hello Linux Inode Practical" > original.txt
ganesh@DESKTOP-79N3772:~/session10$ cat original.txt
Hello Linux Inode Practical
ganesh@DESKTOP-79N3772:~/session10$ ls -li original.txt
26317 original.txt
ganesh@DESKTOP-79N3772:~/session10$ ls -li original.txt
26317 -rw-r--r-- 1 ganesh ganesh 28 Sep 28 08:58 original.txt
ganesh@DESKTOP-79N3772:~/session10$ stat original.txt
  File: original.txt
  size: 28          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 26317         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/  ganesh)   Gid: ( 1000/  ganesh)
Access: 2026-09-28 08:58:50.947133636 +0000
Modify: 2026-09-28 08:58:15.507137121 +0000
Change: 2026-09-28 08:58:15.507137121 +0000
 Birth: 2026-09-28 08:58:15.507137121 +0000
ganesh@DESKTOP-79N3772:~/session10$ find . -name "original.txt"
./original.txt
ganesh@DESKTOP-79N3772:~/session10$
```

Create hard link

```
ganesh@DESKTOP-79N3772:~/session10$ ln original.txt hardlink.txt
ganesh@DESKTOP-79N3772:~/session10$ ls -li original.txt hardlink.txt
26317 -rw-r--r-- 2 ganesh ganesh 28 Sep 28 08:58 hardlink.txt
26317 -rw-r--r-- 2 ganesh ganesh 28 Sep 28 08:58 original.txt
ganesh@DESKTOP-79N3772:~/session10$ cat hardlink.txt
Hello Linux Inode Practical
ganesh@DESKTOP-79N3772:~/session10$
```

Create symbolic link

```

ganesh@DESKTOP-79N3772:~/session10$ cat hardlink.txt
Hello Linux Inode Practical
ganesh@DESKTOP-79N3772:~/session10$ ln -s original.txt symlink.txt
ganesh@DESKTOP-79N3772:~/session10$ ls -li original.txt hardlink.txt symlink.txt
26317 -rw-r--r-- 2 ganesh ganesh 28 Sep 28 08:58 hardlink.txt
26317 -rw-r--r-- 2 ganesh ganesh 28 Sep 28 08:58 original.txt
26319 lrwxrwxrwx 1 ganesh ganesh 12 Sep 28 09:01 symlink.txt -> original.txt
ganesh@DESKTOP-79N3772:~/session10$ cat symlink.txt
Hello Linux Inode Practical
ganesh@DESKTOP-79N3772:~/session10$ _

```

Observation

Inode information was successfully examined, and hard and symbolic links were created and verified.

TASK 2: File Reading and Writing Using `mmap()`

Question

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

Create input file

```

Hello Linux Inode Practical
ganesh@DESKTOP-79N3772:~/session10$ echo "This file is accessed using mmap." > mmapfile.txt
ganesh@DESKTOP-79N3772:~/session10$ cat mmapfile.txt
This file is accessed using mmap.
ganesh@DESKTOP-79N3772:~/session10$ _

```

Create c program

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main()
{
    int fd;
    struct stat st;
    char *data;

    fd = open("mmapfile.txt", O_RDWR);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    if (fstat(fd, &st) == -1)
    {
        perror("fstat");
        close(fd);
    }
}
```

```
ganesh@DESKTOP-79N3772: ~/session10
GNU nano 8.7.1 mmap_demo.c *

data = mmap(NULL, st.st_size,
            PROT_READ | PROT_WRITE,
            MAP_SHARED, fd, 0);

if (data == MAP_FAILED)
{
    perror("mmap");
    close(fd);
    return 1;
}

printf("Original content:\n%s\n", data);

memcpy(data, "Modified using mmap!\n", 21);

msync(data, st.st_size, MS_SYNC);

printf("File modified successfully using mmap.\n");

munmap(data, st.st_size);
close(fd);

return 0;
}
```

Compile

```
ganesh@DESKTOP-79N3772:~/session10$ echo "This file is accessed using mmap." > mmapfile.txt
ganesh@DESKTOP-79N3772:~/session10$ cat mmapfile.txt
This file is accessed using mmap.
ganesh@DESKTOP-79N3772:~/session10$ nano mmap_demo.c
ganesh@DESKTOP-79N3772:~/session10$ gcc mmap_demo.c -o mmap_demo
ganesh@DESKTOP-79N3772:~/session10$ ./mmap_demo
Original content:
This file is accessed using mmap.

File modified successfully using mmap.
ganesh@DESKTOP-79N3772:~/session10$
```

Verify the file

```
ganesh@DESKTOP-79N3772:~/session10$ cat mmapfile.txt
Modified using mmap!
ganesh@DESKTOP-79N3772:~/session10$
```

Observation

File reading and writing were successfully performed using `mmap()`, and the file content was modified through memory mapping.